Leitura — Kotlin para quem vem do Java (Sprint 1, apoio)

Guia de apoio para ler e escrever o Kotlin que aparece no exemplo de Tarefas, tanto na API em Ktor (Web II) quanto no aplicativo em Compose Multiplatform (Dispositivos Móveis). O ponto de partida é o Java que a turma já conhece: cada recurso é apresentado ao lado do equivalente em Java, quando existe um, e depois localizado no código do exemplo.

A leitura dá mais espaço ao que não tem equivalente direto em Java e que mais aparece no curso: lambdas, lambda no fim da chamada, funções de extensão, lambdas com receptor e delegação com `by`. Variáveis, funções e classes entram de forma resumida, como revisão.

Como usar. Cada capítulo aponta uma demonstração: um arquivo curto, com `main`, que roda no Kotlin Playground sem instalar nada. O link abre o código já carregado; o botão **Run** executa. Cada demonstração termina com três experimentos, que são os exercícios desta leitura. Os mesmos arquivos estão em `exemplos/kotlin-playground/` do repositório.

As demonstrações foram compiladas e executadas com Kotlin 2.4.10, a versão do exemplo de Tarefas. Os trechos identificados com o nome de um arquivo (`Rotas.kt`, `Telas.kt`) foram copiados do exemplo.

1. Mapa: do que se vê no código ao nome do recurso

O que aparece no código	Nome do recurso	Capítulo
`val`, `var`, `fun f(x: Int): String`	declarações	2
`= if (...) a else b`, `= when { ... }`	`if` e `when` como expressão	3
`Tarefa?`, `a?.b`, `a ?: c`, `a!!`	nulidade no tipo	4
`data class`, `copy(...)`, `object`, `companion object`	classes	5
`{ x -> ... }`, `{ it.id }`, `(Int) -> Unit`	lambdas e tipos de função	6
`nome { ... }`, `nome(args) { ... }`	lambda no fim da chamada	6.4
`filter { }`, `map { }`, `find { }`	coleções	7
`fun Application.rotas()`, `fun List<Tarefa>.alternando(...)`	função de extensão	8

O que aparece no código	Nome do recurso	Capítulo
`apply { url = ... }`, `routing { get(...) { call... } }`, `Column { Text(...) }`	lambda com receptor	9
`x?.let { }`, `apply`, `also`, `run`, `with`	funções de escopo	9.3
`val r by inject<...>()`, `var texto by remember { ... }`	delegação	10
`receive<NovaTarefa>()`, `"id" to "2"`, `return@Column`	outros símbolos	11
`suspend fun`	corrotinas	12.2

2. Relembrando a escrita básica

```java
// Java
final String titulo = "Estudar";
int total = 0;
total = total + 1;

String situacao(Tarefa t) {
    return t.feita() ? "feita" : "pendente";
}

var t = new Tarefa(1, titulo, false);
System.out.println("Tarefa " + t.id() + ": " + t.titulo());
```

```kotlin
// Kotlin
val titulo = "Estudar"          // val: não aceita nova atribuição (o `final` do Java)
var total = 0                   // var: aceita
total = total + 1

fun situacao(t: Tarefa): String = if (t.feita) "feita" else "pendente"

val t = Tarefa(1, titulo)
println("Tarefa ${t.id}: ${t.titulo}")
```

O que muda:

- O tipo vem depois do nome, separado por dois-pontos: `t: Tarefa`, `fun f(): String`.

- O tipo pode ser omitido quando o compilador o deduz do valor: `val titulo = "Estudar"` é um `String`.
- Não há `new` nem ponto e vírgula.
- Uma função que só devolve uma expressão pode ser escrita com `=`, sem chaves e sem `return`.
- Dentro de um texto, `$nome` insere uma variável e `${expressão}` insere o resultado de uma expressão.
- `Unit` faz o papel do `void`; quando é o tipo de retorno, pode ser omitido.
- `t.titulo` lê a propriedade. Não se escreve `getTitulo()`.
- `==` compara os valores (chama `equals`); `===` compara as referências, como o `==` do Java.

📖 Ref. Kotlin — Basic syntax: https://kotlinlang.org/docs/basic-syntax.html

3. `if`, `when` e `try` devolvem valor

Em Java, `if` é um comando: para guardar o resultado, declara-se a variável antes e atribui-se em cada ramo, ou usa-se o operador ternário. Em Kotlin, `if` é uma expressão: tem um valor, que pode ir direto para um `val`. Por isso a linguagem não tem o operador `? :`.

```
val situacao = if (tarefa.feita) "feita" else "pendente"
```

O `when` substitui o `switch` e também é uma expressão. Sem argumento, ele funciona como uma sequência de `if` / `else if`: o primeiro ramo cuja condição é verdadeira dá o valor.

```
// Tarefa.kt (Web II)
fun violacoes(): List<String> = when {
    titulo.isBlank() -> listOf("titulo: não pode ficar em branco")
    titulo.length > 200 -> listOf("titulo: no máximo 200 caracteres")
    else -> emptyList()
}
```

O `try` segue a mesma regra: `val numero = try { texto.toInt() } catch (e: NumberFormatException) { 0 }`. Kotlin não tem exceções verificadas: nenhuma função é obrigada a declarar `throws`, e nenhuma chamada é obrigada a ter `try`.

📖 Ref. Kotlin — Conditions and loops: https://kotlinlang.org/docs/control-flow.html

▶ Demonstração 1 — expressões e nulidade

4. Nulidade faz parte do tipo

Em Java, toda referência pode ser `null`, e o erro aparece em execução, como `NullPointerException`. Em Kotlin, o tipo diz se o valor pode faltar, e o compilador cobra o tratamento.

Escrita	Significado	Em Java
`Tarefa`	nunca é `null`	sem equivalente; depende de convenção ou de anotações
`Tarefa?`	pode ser `null`	toda referência
`a?.b`	chamada segura: `b` se `a` não for `null`; senão, `null`	`a != null ? a.b : null`
`a ?: c`	operador Elvis: `a` se não for `null`; senão, `c`	`a != null ? a : c`
`a!!`	afirma que não é `null`; se for, lança `NullPointerException`	o uso direto de uma referência

O lado direito do `?:` pode ser um `return` ou um `throw`. É assim que a rota de busca trata os dois casos de erro antes de chegar à resposta:

```
// Rotas.kt (Web II)
val id = call.parameters["id"]?.toIntOrNull()
    ?: throw BadRequestException("O id deve ser um número inteiro")
val tarefa = repositorio.buscar(id)
    ?: throw NotFoundException("A tarefa $id não existe")
call.respond(tarefa)
```

Lendo a primeira instrução da esquerda para a direita: `call.parameters["id"]` devolve um `String?` (o parâmetro pode não existir); `?.toIntOrNull()` só converte se houver texto, e devolve `Int?` (o texto pode não ser um número); `?: throw ...` encerra a rota quando o resultado é `null`. Depois dessa linha, `id` é um `Int`, sem interrogação.

Esse ajuste de tipo se chama *smart cast*: depois de um teste que elimina o `null`, o compilador passa a tratar a variável como não nula.

```
// Telas.kt (Móveis)
if (onVoltar != null) {
    TextButton(onClick = onVoltar) { Text("Voltar") }   // aqui onVoltar é () -> Unit
}
```

O `!!` troca um erro de compilação por um possível erro em execução. Quando ele aparecer, vale procurar uma forma com `?.`, `?:` ou um teste de `null`.

📖 Ref. Kotlin — Null safety: https://kotlinlang.org/docs/null-safety.html

📖 Ref. Kotlin — Nullability in Java and Kotlin: https://kotlinlang.org/docs/java-to-kotlin-nullability-guide.html

▶ Demonstração 1 — expressões e nulidade

5. Classes

5.1 O construtor declara as propriedades

```java
// Java
public record Tarefa(int id, String titulo, boolean feita) {
    public Tarefa(int id, String titulo) {
        this(id, titulo, false);
    }
}
```

```kotlin
// Kotlin
data class Tarefa(val id: Int, val titulo: String, val feita: Boolean = false)
```

Os parênteses depois do nome da classe são o construtor primário. Um parâmetro com `val` ou `var` é, ao mesmo tempo, parâmetro do construtor e propriedade da classe.

`data class` é o parente mais próximo do `record` do Java: o compilador gera `equals`, `hashCode` e `toString` a partir das propriedades do construtor. Gera também `copy`, que cria uma cópia com parte dos valores trocada:

```kotlin
val a = Tarefa(1, "Estudar")
val b = a.copy(feita = true)        // a continua igual; b é outra tarefa
```

5.2 Valores padrão e argumentos nomeados

`val feita: Boolean = false` define um valor padrão: quem chama pode omitir o argumento. É o que em Java se resolve com construtores ou métodos sobrecarregados.

Na chamada, um argumento pode ser passado pelo nome. A ordem deixa de importar, e a leitura fica clara quando há vários parâmetros do mesmo tipo:

```kotlin
// Telas.kt (Móveis)
CartaoTarefa(Tarefa(1, "Estudar Compose", feita = true), onAlternar = {}, onAbrir = {})
```

No Compose, quase toda função tem muitos parâmetros com valor padrão. Por isso as chamadas costumam nomear os argumentos: `Button(onClick = onAdicionar, enabled = valido)`.

5.3 `object` e `companion object`

`object` declara uma classe e a sua única instância, de uma vez. É o padrão *singleton*, feito pela linguagem.

```kotlin
// Banco.kt (Web II): a descrição da tabela existe uma vez só
object Tarefas : Table("tarefas") { ... }

// App.kt (Móveis): a rota da lista não tem argumentos, então basta uma instância
@Serializable
object Lista
```

Kotlin não tem `static`. O que pertence à classe, e não a cada instância, vai num `companion object`, e é chamado pelo nome da classe:

```kotlin
// Banco.kt (Web II)
data class ConfigBanco(val url: String, val usuario: String, val senha: String) {
    companion object {
        fun doAmbiente() = ConfigBanco(...)
    }
}

ConfigBanco.doAmbiente()
```

Funções que não dependem de nenhuma classe podem ficar soltas no arquivo, como `fun main()` e `fun tituloValido(texto: String)`. São as funções de nível superior.

📖 Ref. Kotlin — Data classes: https://kotlinlang.org/docs/data-classes.html

📖 Ref. Kotlin — Object declarations and expressions: https://kotlinlang.org/docs/object-declarations.html

▶ Demonstração 2 — classes

6. Lambdas

6.1 O que é

Uma lambda é um bloco de código guardado num valor: pode ser atribuída a uma variável, passada como argumento e executada depois. Java tem lambdas desde a versão 8; a diferença está no tipo e na escrita.

Em Java, o tipo de uma lambda é uma interface com um único método (`Runnable`, `Predicate<T>`, `Function<T, R>`). Em Kotlin, existe um tipo próprio para funções, escrito `(Entrada) -> Saída`:

Tipo em Kotlin	Interface em Java	Onde aparece no curso
`() -> Unit`	`Runnable`	`onAdicionar`, `onVoltar`
`(Int) -> Unit`	`Consumer<Integer>`	`onAlternar`, `onAbrir`
`(String) -> Unit`	`Consumer<String>`	`onTextoChange`
`() -> T`	`Supplier<T>`	o bloco de `remember { ... }` e o de `lazy { ... }`
`(Tarefa) -> Boolean`	`Predicate<Tarefa>`	a condição de `filter` e de `find`
`(Tarefa) -> String`	`Function<Tarefa, String>`	a transformação de `map`

Para executar, chama-se a variável como se fosse uma função: `onAlternar(2)`.

6.2 A escrita, da forma completa à mais curta

```
val feita: (Tarefa) -> Boolean = { tarefa: Tarefa -> tarefa.feita }   // completa
val feita: (Tarefa) -> Boolean = { tarefa -> tarefa.feita }           // tipo do parâmetro deduzido
val feita: (Tarefa) -> Boolean = { it.feita }                         // um parâmetro só: `it`
```

- As chaves delimitam a lambda. Em Java, a lambda `t -> t.feita()` não tem chaves em volta.
- A seta `->` separa os parâmetros do corpo.
- Quando há exatamente um parâmetro, ele pode ficar sem nome e é chamado de `it`.
- Quando não há parâmetros, não há seta: `{ println("clicou") }`.

Para ler em voz alta, `{ it.feita }` é "dada uma tarefa, a propriedade `feita` dela".

6.3 O valor de uma lambda

O valor de uma lambda é a última expressão do bloco. Não se escreve `return`:

```
val dobro: (Int) -> Int = { numero ->
    val resultado = numero * 2
    resultado                        // este é o valor devolvido
}
```

6.4 Lambda no fim da chamada

Esta regra explica a forma de quase todo o código do curso. Quando o último parâmetro de uma função é uma função, a lambda pode ser escrita depois dos parênteses. Se ela for o único argumento, os parênteses vazios somem. A mesma chamada, em quatro formas:

```
tarefas.filter({ tarefa -> !tarefa.feita })    // 1. lambda como argumento comum
tarefas.filter() { tarefa -> !tarefa.feita }   // 2. lambda depois dos parênteses
tarefas.filter { tarefa -> !tarefa.feita }     // 3. parênteses vazios somem
tarefas.filter { !it.feita }                   // 4. com `it`
```

As quatro compilam para o mesmo programa. A forma 4 é a que se escreve no dia a dia.

Consequência para a leitura: `nome { ... }` e `nome(argumentos) { ... }` são chamadas de função, e o bloco é o último argumento. Não são comandos da linguagem.

```
install(ContentNegotiation) { json() }                 // Ktor: chamada a install, com 2
        argumentos
Button(onClick = onAdicionar) { Text("Adicionar") }    // Compose: chamada a Button, com 2
        argumentos
routing { ... }                                        // Ktor: chamada a routing, com 1
        argumento
```

Uma função pode receber mais de uma lambda. Só a última sai dos parênteses; as outras costumam ser passadas pelo nome:

```
// Telas.kt (Móveis): `key` é uma lambda nomeada; o conteúdo de cada item é a lambda no fim
items(tarefas, key = { it.id }) { tarefa ->
    CartaoTarefa(tarefa, onAlternar = { onAlternar(tarefa.id) }, onAbrir = {
        onAbrir(tarefa.id) })
}
```

6.5 Uma lambda enxerga as variáveis ao redor

Uma lambda pode ler e alterar as variáveis do lugar onde foi criada. Em Java, a lambda só pode ler variáveis que não mudam (`final` ou efetivamente finais).

```
// App.kt (Móveis): as duas lambdas alteram o estado declarado acima delas
val adicionar = {
    tarefas = tarefas.comNova(proximoId, texto)
    proximoId++
    texto = ""
}
val alternar = { id: Int -> tarefas = tarefas.alternando(id) }
```

`adicionar` tem o tipo `() -> Unit`, e `alternar`, o tipo `(Int) -> Unit`. É por isso que podem ser passadas às telas nos parâmetros `onAdicionar` e `onAlternar`.

6.6 Passar uma função que já existe

Uma função declarada com `fun` pode ser usada como valor com `::`. Uma variável que já guarda uma função é passada diretamente, entre parênteses:

```
tarefas.filter(::estaFeita)          // referência a uma função declarada com fun
selecionada?.let(alternar)           // App.kt (Móveis): `alternar` já é um valor (Int) ->
        Unit
```

📖 Ref. Kotlin — Higher-order functions and lambdas: https://kotlinlang.org/docs/lambdas.html

📖 Ref. Kotlin — Passing trailing lambdas: https://kotlinlang.org/docs/lambdas.html#passing-trailing-lambdas

▶ Demonstração 3 — lambdas

7. Coleções

As operações que em Java pedem `stream()` e um coletor são funções diretas da lista, e recebem uma lambda no fim da chamada.

```java
// Java
List<String> pendentes = tarefas.stream()
    .filter(t -> !t.feita())
    .map(t -> t.titulo())
    .toList();
```

```kotlin
// Kotlin
val pendentes = tarefas.filter { !it.feita }.map { it.titulo }
```

Função	Devolve
`filter { condição }`	uma lista com os elementos que passam na condição
`map { transformação }`	uma lista com cada elemento transformado
`find { condição }`	o primeiro elemento que passa, ou `null`
`any { condição }`	`true` se algum elemento passa
`count { condição }`	quantos passam
`sortedBy { chave }`	uma lista ordenada pela chave
`lista + elemento`	uma lista com o elemento a mais

`List` é somente leitura: não tem `add` nem `remove`. Para alterar no lugar, o tipo é `MutableList`, criado com `mutableListOf(...)`. O curso usa listas somente leitura, e "alterar" significa criar uma lista nova:

```kotlin
// Tarefa.kt (Móveis): devolve outra lista, com uma tarefa trocada por uma cópia
fun List<Tarefa>.alternando(id: Int): List<Tarefa> =
    map { if (it.id == id) it.copy(feita = !it.feita) else it }
```

No Compose isso tem uma consequência prática: a tela se redesenha quando o estado recebe um valor novo (`tarefas = tarefas.alternando(id)`). Alterar um objeto por dentro não avisa ninguém.

📖 Ref. Kotlin — Collections overview: https://kotlinlang.org/docs/collections-overview.html

📖 Ref. Kotlin — Collections in Java and Kotlin: https://kotlinlang.org/docs/java-to-kotlin-collections-guide.html

▶ Demonstração 4 — coleções

8. Funções de extensão

8.1 O que é

Uma função de extensão acrescenta uma função a um tipo que já existe, sem herança e sem alterar a classe. O tipo é escrito antes do nome da função, separado por um ponto, e se chama receptor.

```java
// Java: método estático numa classe utilitária
class Tarefas {
    static List<Tarefa> alternando(List<Tarefa> lista, int id) { ... }
}

Tarefas.alternando(lista, 2);
```

```kotlin
// Kotlin: List<Tarefa> é o receptor
fun List<Tarefa>.alternando(id: Int): List<Tarefa> =
    map { if (it.id == id) it.copy(feita = !it.feita) else it }

lista.alternando(2)
```

Dentro da função, `this` é o receptor: a lista sobre a qual a função foi chamada. O `this` pode ser omitido; `map { ... }` ali quer dizer `this.map { ... }`.

A chamada fica com a forma de um método (`lista.alternando(2)`), o que permite encadear:

```kotlin
val depois = tarefas.comNova(3, "Gravar o vídeo").alternando(1)
```

8.2 O que ela não faz

- Não altera a classe. É uma função comum; o compilador a transforma numa chamada que recebe o receptor como primeiro argumento.
- Não enxerga os membros privados do tipo. Só usa o que é público.
- Não substitui um membro: se a classe já tem uma função com o mesmo nome e os mesmos parâmetros, a da classe é a escolhida.
- Precisa ser importada quando está em outro pacote. É por isso que o `Rotas.kt` importa `io.ktor.server.routing.get` e `io.ktor.server.response.respond`: são extensões.

8.3 No exemplo de Tarefas

```
// Aplicacao.kt (Web II): extensões de Application organizam a configuração do servidor
fun Application.modulo(config: ConfigBanco = ConfigBanco.doAmbiente()) { ... }
fun Application.configurar(repositorio: RepositorioDeTarefas) {
    install(Koin) { ... }          // this.install(...): o this é a Application
    tratarErros()                  // outra extensão de Application, em Erros.kt
    rotas()                        // outra extensão de Application, em Rotas.kt
}

// Banco.kt (Web II): extensão privada, visível só neste arquivo
private fun ResultRow.paraTarefa() = Tarefa(this[Tarefas.id], this[Tarefas.titulo],
        this[Tarefas.feita])
```

Em `embeddedServer(CIO, port = 8080) { modulo() }`, o bloco é executado com a aplicação como `this` (capítulo 9), e por isso `modulo()` pode ser chamado sem nada antes.

Grande parte da biblioteca padrão é feita de extensões: `filter` e `map` são extensões de coleções; `isBlank` e `toIntOrNull`, de `String`. No Compose, `Modifier.padding(16.dp)` é uma extensão de `Modifier`, e `16.dp` é uma propriedade de extensão de `Int`.

📖 Ref. Kotlin — Extensions: https://kotlinlang.org/docs/extensions.html

▶ Demonstração 5 — funções de extensão

9. Lambda com receptor

9.1 O que é

Este recurso junta os capítulos 6 e 8: é uma lambda que, por dentro, tem um `this`. O tipo se escreve com o receptor antes dos parênteses: `Conexao.() -> Unit`.

```
// Lambda comum: (Conexao) -> Unit. O objeto chega como parâmetro e precisa ser citado.
configurar { conexao ->
    conexao.url = "jdbc:postgresql://localhost/tarefas"
    conexao.usuario = "tarefas"
}

// Lambda com receptor: Conexao.() -> Unit. O objeto é o this do bloco.
configurar {
    url = "jdbc:postgresql://localhost/tarefas"       // this.url
    usuario = "tarefas"                               // this.usuario
}
```

Quem define qual das duas formas vale é a função chamada, no tipo do parâmetro:

```
fun configurar(bloco: Conexao.() -> Unit): Conexao {
    val conexao = Conexao()
    conexao.bloco()                 // executa o bloco com `conexao` como this
    return conexao
}
```

Java não tem esse recurso. O parente mais próximo é o padrão *builder* com chamadas encadeadas (`config.setUrl(...).setUsuario(...)`).

Ao ler um bloco, a pergunta útil é: quem é o `this` aqui? A resposta está na assinatura da função chamada. Na IDE, basta apontar para o nome da função, ou ativar as dicas que mostram `this: Tipo` no início de cada bloco.

9.2 Blocos dentro de blocos

Uma biblioteca que oferece várias funções desse tipo permite escrever uma estrutura inteira com blocos aninhados. É o que a documentação chama de *type-safe builder* ou DSL (linguagem específica de domínio). O Ktor usa essa técnica para rotas, e o Compose, para telas; os capítulos 12 e 13 leem os dois casos linha a linha.

Dentro de um bloco aninhado, continuam visíveis os `this` dos blocos de fora. É por isso que, dentro de `get("/{id}") { ... }`, o código consegue usar `repositorio`, declarado em `Application.rotas()`.

9.3 Funções de escopo

A biblioteca padrão tem cinco funções que executam um bloco sobre um objeto. Diferem em duas coisas: como o objeto aparece dentro do bloco (`it` ou `this`) e o que a chamada devolve.

Função	O objeto é	Devolve	Uso típico
`let`	`it`	o resultado do bloco	fazer algo só quando o valor não é `null` : `x?.let { ... }`
`apply`	`this`	o próprio objeto	configurar um objeto recém-criado
`also`	`it`	o próprio objeto	um efeito a mais, como um registro em log
`run`	`this`	o resultado do bloco	calcular algo a partir do objeto
`with(x)`	`this`	o resultado do bloco	várias chamadas sobre o mesmo objeto

Os dois casos do exemplo de Tarefas:

```
// Banco.kt (Web II): cria, configura e devolve o HikariConfig, numa expressão só
HikariDataSource(
    HikariConfig().apply {
        jdbcUrl = config.url
        username = config.usuario
        password = config.senha
        maximumPoolSize = 5
    },
)

// App.kt (Móveis): `selecionada` é Int?. A função só é chamada quando há uma tarefa
        selecionada.
onAlternar = { selecionada?.let(alternar) }
```

Para começar, `let` e `apply` cobrem a maior parte dos casos. Blocos de escopo aninhados ficam difíceis de ler; quando isso acontecer, uma variável com nome resolve melhor.

📖 Ref. Kotlin — Function literals with receiver: https://kotlinlang.org/docs/lambdas.html#function-literals-with-receiver

📖 Ref. Kotlin — Scope functions: https://kotlinlang.org/docs/scope-functions.html

📖 Ref. Kotlin — Type-safe builders: https://kotlinlang.org/docs/type-safe-builders.html

▶ Demonstração 6 — lambda com receptor e funções de escopo

10. Delegação com `by`

`by` entrega a outro objeto, o delegado, o trabalho de ler e gravar uma propriedade. O compilador troca cada leitura por uma chamada a `getValue()` do delegado e cada gravação por uma chamada a `setValue()`. Java não tem equivalente.

O caso mais simples é o `by lazy`, da biblioteca padrão: o valor é calculado no primeiro uso e guardado.

```
val repositorio: Repositorio by lazy { Repositorio() }     // ainda não criou nada
repositorio.listar()                                       // cria aqui, uma vez
```

No Ktor com Koin, a dependência é buscada no primeiro uso:

```
// Rotas.kt (Web II)
val repositorio by inject<RepositorioDeTarefas>()
```

`inject` devolve um `Lazy<RepositorioDeTarefas>`. Sem o `by`, cada uso seria `repositorio.value.listar()`. Com o `by`, a variável se comporta como um `RepositorioDeTarefas`.

No Compose, o delegado é o estado observável:

```
// Sem by: `texto` é um MutableState<String>
val texto = remember { mutableStateOf("") }
texto.value = "Estudar"

// Com by: `texto` se comporta como um String
var texto by remember { mutableStateOf("") }
texto = "Estudar"
```

É nas chamadas a `getValue()` e `setValue()` que o Compose registra quem leu o estado e avisa quando ele mudou. Dois detalhes práticos: com `by`, a variável precisa ser `var`, porque há gravação; e o arquivo precisa dos imports `androidx.compose.runtime.getValue` e `androidx.compose.runtime.setValue` (a IDE sugere os dois).

📖 Ref. Kotlin — Delegated properties: https://kotlinlang.org/docs/delegated-properties.html

📖 Ref. Android — State and Jetpack Compose: https://developer.android.com/develop/ui/compose/state

▶ Demonstração 9 — delegação

11. Outros símbolos que aparecem no exemplo

No código	O que é	Em Java
`call.receive<NovaTarefa>()`, `inject<RepositorioDeTarefas>()`, `composable<Detalhe> { }`	o tipo é passado entre `< >`, na chamada	um argumento `NovaTarefa.class`
`"id" to "2"`, `Tarefas.id eq id`	função infixa: `a to b` é `a.to(b)`	chamada comum de método
`call.parameters["id"]`, `this[Tarefas.titulo]`, `it[titulo] = nova.titulo`	operador de índice: `a[i]` chama `a.get(i)`; `a[i] = v` chama `a.set(i, v)`	só existe para arrays
`val (id, titulo) = tarefa`	desestruturação: uma variável para cada propriedade da `data class`	sem equivalente
`1..3`, `0 until n`	intervalos, usados em `for (i in 1..3)`	`for (int i = 1; i <= 3; i++)`
`return@Column`	sai só do bloco passado a `Column`, não da função inteira	sem equivalente
`fun `id inexistente devolve 404`()`	nome entre crases: aceita espaços; usado em nomes de teste	sem equivalente
`@OptIn(ExperimentalKtorApi::class)`	aceita o uso de uma API marcada como experimental	sem equivalente direto
`Tarefa::class`	a referência à classe	`Tarefa.class`

Sobre o `return@Column`, que aparece em `TelaDetalhe`:

```
// Telas.kt (Móveis)
Column(modifier.padding(16.dp)) {
    if (tarefa == null) {
        Text("Selecione uma tarefa")
        return@Column               // encerra o bloco do Column; nada abaixo é executado
    }
    Text(tarefa.titulo, ...)        // daqui em diante, tarefa não é null
}
```

📖 Ref. Kotlin — Returns and jumps: https://kotlinlang.org/docs/returns.html#return-to-labels

📖 Ref. Kotlin — Reified type parameters: https://kotlinlang.org/docs/inline-functions.html#reified-type-parameters

📖 Ref. Kotlin — Infix notation: https://kotlinlang.org/docs/functions.html#infix-notation

12. No Ktor: lendo o `Rotas.kt`

12.1 As rotas

```kotlin
// Rotas.kt (Web II), sem a parte do OpenAPI
fun Application.rotas() {                                   // (a)
    val repositorio by inject<RepositorioDeTarefas>()       // (b)

    routing {                                               // (c)
        route("/tarefas") {                                 // (d)
            get { call.respond(repositorio.listar()) }      // (e)

            post {
                val nova = call.receive<NovaTarefa>()       // (f)
                val violacoes = nova.violacoes()
                if (violacoes.isNotEmpty()) throw EntradaInvalida(violacoes)
                val criada = repositorio.adicionar(nova)
                call.response.header(HttpHeaders.Location, "/tarefas/${criada.id}")
                call.respond(HttpStatusCode.Created, criada)
            }
        }
    }
}
```

- (a) Função de extensão de `Application`. Dentro dela, `this` é a aplicação.
- (b) Delegação: o repositório vem do Koin, no primeiro uso.
- (c) `routing` é uma função, e o bloco é o seu único argumento (lambda no fim da chamada). O bloco é uma lambda com receptor: dentro dele, `this` é o objeto que registra rotas.
- (d) `route("/tarefas") { ... }` é outra chamada, com dois argumentos: o caminho e o bloco.
- (e) `get { ... }` registra uma rota. O bloco não é executado agora: fica guardado e roda a cada requisição. Dentro dele, o `this` é um `RoutingContext`, que tem a propriedade `call`. É por isso que `call` pode ser usado sem ter sido declarado.
- (f) O tipo entre `< >` diz em que classe o JSON deve ser convertido.

O mesmo começo, sem as abreviações da linguagem:

```
this.routing({
    this.route("/tarefas", {
        this.get({ this.call.respond(repositorio.listar()) })
    })
})
```

Como as rotas são chamadas de função comuns, cabem entre elas `if`, laços e funções auxiliares. No Quarkus, as mesmas rotas são declaradas por anotações (`@GET`, `@Path`).

A demonstração 7 escreve um Ktor em miniatura, com `routing`, `get` e `call`, em cerca de trinta linhas. Ela mostra que essa sintaxe é feita com recursos comuns da linguagem.

12.2 `suspend`

```
// Tarefa.kt (Web II)
interface RepositorioDeTarefas {
    suspend fun listar(): List<Tarefa>
    suspend fun buscar(id: Int): Tarefa?
    suspend fun adicionar(nova: NovaTarefa): Tarefa
}
```

`suspend` marca uma função que pode pausar e continuar depois. Enquanto ela espera a resposta do banco, a thread fica livre para atender outras requisições. A unidade que executa esse código se chama corrotina; muitas corrotinas se revezam em poucas threads.

A regra que o compilador cobra: uma função `suspend` só pode ser chamada de outra função `suspend` ou de dentro de uma corrotina. No Ktor, o bloco de cada rota já é `suspend`, e cada requisição já roda numa corrotina. Por isso `repositorio.buscar(id)` é chamado diretamente, com a aparência de código sequencial.

Em Java, os caminhos para o mesmo resultado são `CompletableFuture` ou threads virtuais. O Quarkus do exemplo segue outra estratégia: executa o método numa thread de trabalho.

📖 Ref. Ktor — Routing: https://ktor.io/docs/server-routing.html

📖 Ref. Kotlin — Coroutines basics: https://kotlinlang.org/docs/coroutines-basics.html

▶ Demonstração 7 — um Ktor em miniatura · ▶ Demonstração 10 — corrotinas

13. No Compose: lendo o `Telas.kt`

13.1 Os parâmetros de uma tela

```
// Telas.kt (Móveis)
@Composable
fun TelaDetalhe(
    tarefa: Tarefa?,                 // (a)
    onAlternar: () -> Unit,          // (b)
    onVoltar: (() -> Unit)?,         // (c)
    modifier: Modifier = Modifier,   // (d)
)
```

- (a) Tipo anulável: pode não haver tarefa selecionada.
- (b) Tipo de função sem parâmetros e sem resultado. A tela avisa que o usuário tocou; quem chamou decide o que fazer. Dados entram por parâmetros comuns, e eventos saem por parâmetros de tipo função.
- (c) Função opcional. Os parênteses externos são necessários: `(() -> Unit)?` é "uma função, ou `null`"; `() -> Unit?` seria "uma função que devolve `Unit` ou `null`".
- (d) Valor padrão: quem chama pode omitir. O valor `Modifier` é um `companion object` da interface `Modifier`, que representa "nenhum ajuste".

Na lista, `onAlternar` é `(Int) -> Unit`, porque a lista precisa dizer qual tarefa foi tocada. No cartão, é `() -> Unit`, porque o cartão já é de uma tarefa. A ligação entre os dois é uma lambda:

```
CartaoTarefa(tarefa, onAlternar = { onAlternar(tarefa.id) }, onAbrir = { onAbrir(tarefa.id)
        })
```

13.2 A árvore de componentes

```
// Telas.kt (Móveis), resumido
Column(modifier.padding(16.dp)) {                                    // (a)
    Text("Minhas tarefas")
    FormularioTarefa(texto, onTextoChange, tituloValido(texto), onAdicionar)
    LazyColumn(verticalArrangement = Arrangement.spacedBy(8.dp)) {   // (b)
        items(tarefas, key = { it.id }) { tarefa ->                  // (c)
            CartaoTarefa(tarefa, ...)
        }
    }
}
```

- (a) `Column` é uma função, não uma classe. Recebe dois argumentos: o `modifier` e o bloco de conteúdo (lambda no fim da chamada). Dentro do bloco, `this` é um `ColumnScope`.
- (b) `LazyColumn` recebe um argumento nomeado e o bloco. Dentro dele, `this` é um `LazyListScope`.
- (c) `items` é uma função do `LazyListScope`: só existe dentro do bloco de uma lista. Recebe a lista, uma lambda nomeada (`key`) e, no fim, a lambda que desenha cada item.

O receptor também limita o que pode ser chamado em cada lugar. `Modifier.weight(1f)`, usado em `Row` e `Column` para dividir o espaço, é uma função do `RowScope` e do `ColumnScope`: fora desses blocos, não compila.

Como a tela é feita de chamadas de função, cabem nela `if` e `for`:

```
if (onVoltar != null) {
    TextButton(onClick = onVoltar) { Text("Voltar") }
}
```

A demonstração 8 escreve um Compose em miniatura, que monta uma árvore de componentes com essa mesma escrita. O Compose de verdade funciona de outro jeito por dentro: a anotação `@Composable` ativa um plugin do compilador, que acompanha o estado e redesenha só o que mudou.

13.3 O estado

```
// App.kt (Móveis)
var tarefas by remember { mutableStateOf(tarefasIniciais) }     // delegação (capítulo 10)
val alternar = { id: Int -> tarefas = tarefas.alternando(id) }  // lambda que reatribui o
        estado
```

Três recursos numa linha cada: `by` faz `tarefas` se comportar como uma lista; a lambda enxerga e altera essa variável (seção 6.5); e `alternando` é uma extensão que devolve uma lista nova (capítulo 8). A reatribuição é o que faz a tela se redesenhar.

📖 Ref. Android — Kotlin for Jetpack Compose: https://developer.android.com/develop/ui/compose/kotlin

▶ Demonstração 8 — um Compose em miniatura

14. Escrever de modo idiomático

Código traduzido do Java linha a linha compila em Kotlin, mas fica mais longo do que precisa. A tabela reúne as trocas mais comuns:

Hábito trazido do Java	Em Kotlin
declarar tudo com `var`	`val` por padrão; `var` só quando o valor muda
variável declarada antes do `if` e atribuída em cada ramo	`val x = if (...) a else b`
`"Tarefa " + id + ": " + titulo`	`"Tarefa $id: $titulo"`
classe com campos privados, construtor e getters	`data class` com as propriedades no construtor
construtores ou métodos sobrecarregados	parâmetros com valor padrão
`if (x != null) { x.f() }`	`x?.f()`
`x != null ? x : padrao`	`x ?: padrao`
laço com índice e `lista.get(i)`	`for (item in lista)`, ou `filter` e `map`
classe `AlgumaCoisaUtil` com métodos estáticos	funções de nível superior ou de extensão
`lista.stream().filter(...).toList()`	`lista.filter { ... }`
`x.equals(y)`	`x == y`
`!!` para "resolver" um erro de nulidade	`?.`, `?:` ou um teste de `null`

A demonstração 11 traz o mesmo programa nas duas versões, lado a lado.

O Android Studio e o IntelliJ IDEA sugerem boa parte dessas trocas: um trecho sublinhado com uma lâmpada ao lado costuma ter uma forma mais curta, aplicada com `Alt+Enter`.

📖 Ref. Kotlin — Idioms: https://kotlinlang.org/docs/idioms.html

📖 Ref. Kotlin — Coding conventions: https://kotlinlang.org/docs/coding-conventions.html

📖 Ref. Kotlin — Comparison to Java: https://kotlinlang.org/docs/comparison-to-java.html

▶ Demonstração 11 — idiomático

15. Exercícios

Cada demonstração termina com três experimentos, escritos como comentários no fim do `main`. A sugestão é fazer na ordem:

	Demonstração	Recursos
1	▶ Expressões e nulidade	`if`, `when`, `?.`, `?:`
2	▶ Classes	`data class`, `copy`, `object`, `companion object`
3	▶ Lambdas	tipos de função, `it`, lambda no fim da chamada

	Demonstração	Recursos
4	▶ Coleções	`filter`, `map`, `find`, listas somente leitura
5	▶ Funções de extensão	receptor, `this`, propriedade de extensão
6	▶ Lambda com receptor	`Tipo.() -> Unit`, `apply`, `let`, `also`, `run`, `with`
7	▶ Um Ktor em miniatura	extensão, lambda com receptor e lambda no fim, juntas
8	▶ Um Compose em miniatura	o mesmo, numa árvore de componentes com eventos
9	▶ Delegação	`by`, `by lazy`, um delegado que observa mudanças
10	▶ Corrotinas	`suspend`, `async`, dez mil corrotinas
11	▶ Idiomático	o mesmo programa em estilo Java e em estilo Kotlin

Para praticar no código do grupo:

1. Abra um arquivo do projeto e, para cada bloco `{ ... }`, responda: de qual função ele é argumento? Dentro dele, o objeto é `it` ou `this`? De que tipo?
2. Reescreva uma chamada com lambda no fim na forma sem abreviações (como nos capítulos 12 e 13) e confira que compila.
3. Encontre uma função do projeto que receba uma lista e devolva outra, e transforme-a numa função de extensão.
4. Procure todos os `!!` do projeto e substitua cada um por `?.`, `?:` ou um teste de `null`.

16. Perguntas frequentes

Preciso dominar tudo isto para entregar a Sprint 1? Não. Para ler e adaptar o exemplo de Tarefas, os capítulos 4, 6, 8 e 9 são os que mais ajudam. Os demais servem de consulta.

Quando uso `it` e quando dou nome ao parâmetro? `it` funciona bem em blocos de uma linha. Em blocos maiores, ou quando há um bloco dentro de outro, um nome (`{ tarefa -> ... }`) evita dúvida sobre a quem o `it` se refere.

Como descubro quem é o `this` de um bloco? Pela assinatura da função chamada: o tipo do último parâmetro tem a forma `Tipo.() -> ...`. No Android Studio e no IntelliJ IDEA, aponte o cursor para o nome da função, ou use `Ctrl+P` dentro dos parênteses.

Por que `routing`, `get`, `Column` e `Button` não aparecem na lista de palavras da linguagem? Porque são funções de biblioteca. O que a linguagem oferece é a regra que permite escrever o último argumento fora dos parênteses.

Função de extensão ou função comum? Extensão quando a função é, na leitura, uma operação sobre um tipo (`lista.alternando(id)`). Função comum quando não há um objeto principal

(`tituloValido(texto)`).

Posso misturar Java e Kotlin no mesmo projeto? Pode: os dois compilam para a JVM, e um chama o outro. As bibliotecas Java usadas no exemplo (HikariCP, Flyway) são chamadas do Kotlin sem adaptação.

O Playground roda o código do exemplo de Tarefas? Não: ele não tem Ktor nem Compose. Por isso as demonstrações 7 e 8 usam versões em miniatura, escritas só com a linguagem. A biblioteca `kotlinx.coroutines`, usada na demonstração 10, está disponível lá.

Referências

Todas as páginas foram consultadas em outubro de 2026. A documentação do Kotlin e a do Android são publicadas sob a licença Apache 2.0.

Para estudar em sequência

- Kotlin — Tour of Kotlin (com código executável e exercícios): https://kotlinlang.org/docs/kotlin-tour-welcome.html
- Kotlin — Koans (exercícios no navegador, para quem vem do Java): https://play.kotlinlang.org/koans
- Android — Learn the Kotlin programming language: https://developer.android.com/kotlin/learn
- Android — Android Basics with Compose (unidades 1 a 3 têm trilhas de Kotlin): https://developer.android.com/courses/android-basics-compose/course
- JetBrains — Kotlin for Education (curso Programming in Kotlin, com slides): https://kotlinlang.org/education/

Kotlin e Java

- Kotlin — Comparison to Java: https://kotlinlang.org/docs/comparison-to-java.html
- Kotlin — Strings in Java and Kotlin: https://kotlinlang.org/docs/java-to-kotlin-idioms-strings.html
- Kotlin — Collections in Java and Kotlin: https://kotlinlang.org/docs/java-to-kotlin-collections-guide.html
- Kotlin — Nullability in Java and Kotlin: https://kotlinlang.org/docs/java-to-kotlin-nullability-guide.html
- Kotlin — Calling Java from Kotlin: https://kotlinlang.org/docs/java-interop.html

Linguagem

- Kotlin — Basic syntax: https://kotlinlang.org/docs/basic-syntax.html
- Kotlin — Conditions and loops: https://kotlinlang.org/docs/control-flow.html
- Kotlin — Null safety: https://kotlinlang.org/docs/null-safety.html
- Kotlin — Classes: https://kotlinlang.org/docs/classes.html
- Kotlin — Data classes: https://kotlinlang.org/docs/data-classes.html

- Kotlin — Object declarations and expressions: https://kotlinlang.org/docs/object-declarations.html
- Kotlin — Functions: https://kotlinlang.org/docs/functions.html
- Kotlin — Higher-order functions and lambdas: https://kotlinlang.org/docs/lambdas.html
- Kotlin — Collections overview: https://kotlinlang.org/docs/collections-overview.html
- Kotlin — Extensions: https://kotlinlang.org/docs/extensions.html
- Kotlin — Scope functions: https://kotlinlang.org/docs/scope-functions.html
- Kotlin — Type-safe builders: https://kotlinlang.org/docs/type-safe-builders.html
- Kotlin — Delegated properties: https://kotlinlang.org/docs/delegated-properties.html
- Kotlin — Returns and jumps: https://kotlinlang.org/docs/returns.html
- Kotlin — Idioms: https://kotlinlang.org/docs/idioms.html
- Kotlin — Coding conventions: https://kotlinlang.org/docs/coding-conventions.html
- Kotlin — Coroutines basics: https://kotlinlang.org/docs/coroutines-basics.html

Ktor e Compose

- Ktor — Routing: https://ktor.io/docs/server-routing.html
- Koin — Ktor: https://insert-koin.io/docs/reference/koin-ktor/ktor
- Android — Kotlin for Jetpack Compose: https://developer.android.com/develop/ui/compose/kotlin
- Android — State and Jetpack Compose: https://developer.android.com/develop/ui/compose/state

Ferramenta

- Kotlin Playground: https://play.kotlinlang.org/
- Kotlin — Run code snippets: https://kotlinlang.org/docs/run-code-snippets.html